

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

Гомельский государственный технический университет имени П.О.Сухого

Факультет автоматизированных и информационных систем

Кафедра «Информационные технологии»

специальность 6-05-0611-01 «Информационные системы и технологии»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовой работе

по дисциплине «Распределённые информационные системы»

на тему: Программный продукт для клиент-серверной передачи топологии
электрических сетей через *REST-API* на базе *Django*

Исполнитель: студент группы ИТП-31
Стасев Б.В.

Руководитель: доцент
Комраков В.В.

Дата проверки:

Дата допуска к защите:

Оценка работы:

Подписи членов комиссии по защите
Курсовой работы:

СОДЕРЖАНИЕ

Введение.....	4
1 Обзор источников для разработки <i>Web</i> -приложения	5
1.1 Обзор существующих решений для проектирование электрических сетей.....	5
1.2 Обзор архитектурных стилей и протоколов клиент-серверного взаимодействия	6
1.3 Программные средства для реализации программного продукта	8
2 Проектирование клиент-серверного приложения и <i>REST-API</i>	12
2.1 Архитектура клиент-серверного приложения	12
2.2 Проектирование <i>REST-API</i>	13
2.3 Диаграмма классов приложения.....	15
2.4 Диаграмма последовательности обмена по <i>REST-API</i>	16
2.5 Проектирование модели данных для хранения передаваемой топологии	17
2.6 Проектирование клиентского интерфейса	20
3 Реализаци и тестирование <i>REST-API</i>	23
3.1 Особенности реализации серверной части и <i>REST-API</i>	23
3.2 Функциональное тестирование <i>REST-API</i>	24
3.3 <i>Unit</i> -тесты.....	25
Заключение	27
Список использованных источников	28
Приложение А Структура проекта.....	29
Приложение Б Результаты тестирования <i>REST-API</i>	30
Приложение В Листинг программы.....	34

ВВЕДЕНИЕ

Современная электроэнергетика характеризуется ростом числа распределительных сетей 0,4 кВ и усложнением их топологии. В составе одной сети находятся десятки подстанций и тысячи абонентов, информация о которых должна обрабатываться централизованно и быть доступна сотрудникам с разных рабочих мест. Текущая практика хранения схем в виде разрозненных файлов САПР не позволяет организовать совместную работу и удалённый доступ к актуальной топологии, таким образом тема является актуальной.

Решением является построение программного продукта в клиент-серверной архитектуре, где графический редактор в браузере и серверная часть взаимодействуют через интерфейс *REST*. Архитектурный стиль *REST*, использующий методы протокола *HTTP* и формат *JSON*, обеспечивает независимость клиента от технологий сервера, прозрачность кэширования и удобство интеграции. Этим обусловлена актуальность темы курсовой работы.

Целью курсовой работы является разработка программного продукта *VoltStream* для клиент-серверной передачи топологии сетей 0,4 кВ через *REST-API* на базе фреймворка *Django*.

Для достижения цели необходимо решить следующие задачи:

- проанализировать программные решения и архитектурные стили взаимодействия (*REST*, *SOAP*, *GraphQL*, *gRPC*, *WebSocket*);
- обосновать выбор языка, веб-фреймворка, СУБД, клиентских технологий и средств разработки;
- спроектировать архитектуру приложения и *REST-API*, определив состав ресурсов, методы *HTTP* и формат *JSON*-сообщений;
- разработать модель данных для сериализации топологии схемы в *JSON* и её десериализации на сервере;
- реализовать на *Django* функционал построения схем 0,4 кВ, управление справочниками и версионирование через *REST-API*;
- провести функциональное и модульное тестирование разработанного *REST-API*.

Объектом разработки является программный продукт для клиент-серверной передачи топологии электрических сетей. Предметом исследования – архитектура веб-приложения и состав *REST-API*, обеспечивающего передачу схемы между браузером и сервером *Django*.

1 ОБЗОР ИСТОЧНИКОВ ДЛЯ РАЗРАБОТКИ WEB-ПРИЛОЖЕНИЯ

1.1 Обзор существующих решений для проектирование электрических сетей

На рынке инженерного программного обеспечения для проектирования электрических сетей представлено несколько крупных продуктов, каждый из которых имеет свои сильные и слабые стороны. К числу наиболее распространённых решений можно отнести *AutoCAD Electrical* [1], *RastrWin3* [2], *EnergyCS Электрика* [3], *DigSilent PowerFactory* [4] и *Eaton Power Xpert*. Указанные продукты позволяют строить схемы электрических сетей и выполнять расчёт установившихся режимов, однако их применение ограничено рядом существенных недостатков.

AutoCAD Electrical является промышленным стандартом для черчения однолинейных схем, но не имеет встроенной базы данных топологии сети – все объекты сохраняются как графические примитивы внутри *DWG*-файла. Это приводит к дублированию параметров одного и того же оборудования в разных схемах и существенно затрудняет анализ. *RastrWin3* и *DigSilent PowerFactory* ориентированы преимущественно на сети высокого напряжения и расчёт режимов, при этом построение распределительных сетей 0,4 кВ в них реализовано неудобно. *EnergyCS Электрика*, напротив, ориентирован именно на сети 0,4 кВ, однако является закрытой и сравнительно дорогой настольной системой и не поддерживает совместную работу через сеть.

В таблице 1.1 приведён сравнительный анализ существующих программных решений по следующим критериям: ориентация на сети 0,4 кВ, способ обмена данными между клиентом и сервером, доступ через веб-интерфейс, поддержка многопользовательской работы и стоимость лицензии.

Таблица 1.1 – Сравнение существующих программных решений

Продукт	Сети 0,4 кВ	Обмен данными	Веб-доступ	Многопользовательский режим	Стоимость лицензии
1	2	3	4	5	6
<i>AutoCAD Electrical</i>	Частично	Локальные <i>DWG</i> -файлы	Нет	Нет	Высокая
<i>RastrWin3</i>	Слабо	Локальные <i>.rg2</i> -файлы	Нет	Нет	Средняя

Продолжение таблицы 1.1

1	2	3	4	5	6
<i>EnergyCS</i> Электрика	Да	Локальные файлы	Нет	Нет	Высокая
<i>DigSilent PowerFactory</i>	Слабо	Локальные файлы	Нет	Нет	Очень высокая
<i>Eaton Power Xpert</i>	Да	Закрытый протокол	Да	Да	Очень высокая
<i>VoltStream</i> (предлагаемое)	Да	<i>REST-API + JSON</i>	Да	Да	Бесплатно
<i>EnergyCS</i> Электрика	Да	Локальные файлы	Нет	Нет	Высокая

Анализ таблицы 1.1 показывает, что существующие решения либо хранят топологию сети в локальных файлах САПР без поддержки сетевого обмена, либо используют закрытые проприетарные протоколы взаимодействия клиента и сервера. Это исключает интеграцию с внешними системами и требует приобретения дорогостоящих лицензий. Бесплатных программных решений с открытым *REST-API*, ориентированных на распределительные сети 0,4 кВ, на момент работы не выявлено. Это обосновывает целесообразность разработки собственного продукта *VoltStream*.

1.2 Обзор архитектурных стилей и протоколов клиент-серверного взаимодействия

Под клиент-серверным приложением в данной работе понимается программное обеспечение, в котором клиентская часть (графический интерфейс) и серверная часть (бизнес-логика и хранилище данных) разнесены на разные узлы сети и взаимодействуют между собой по сетевому протоколу [5]. Ключевым архитектурным решением такого приложения является выбор стиля интеграции – способа, которым клиент и сервер обмениваются данными.

К рассмотрению выбраны четыре наиболее распространённых архитектурных стиля интеграции:

- *REST (Representational State Transfer)* – стиль, при котором ресурсы предметной области адресуются *URI*, а действия над ними выполняются стандартными методами *HTTP (GET, POST, PUT, DELETE)*; представления ресурсов передаются обычно в формате *JSON*;

- *SOAP (Simple Object Access Protocol)* – *XML*-протокол удалённого вызова процедур со строгой схемой *WSDL* и расширенными возможностями безопасности (*WS-Security*);

- *GraphQL* – язык запросов к данным, при котором клиент сам описывает состав возвращаемых полей одним *POST*-запросом к единственному эндпоинту;
- *gRPC* – протокол вызова удалённых процедур поверх *HTTP/2* с бинарной сериализацией *Protocol Buffers* и стриминговыми вызовами;
- *WebSocket* – двунаправленный полнодуплексный канал поверх одного *TCP*-соединения, ориентированный на обмен сообщениями реального времени.

В таблице 1.2 приведено сопоставление перечисленных стилей по основным характеристикам, значимым для передачи топологии электрической сети.

Таблица 1.2 – Сравнение архитектурных стилей клиент-серверного взаимодействия

Стиль	Транспорт	Формат данных	Сложность реализации	Поддержка в <i>Django</i>
<i>REST</i>	<i>HTTP/1.1</i>	<i>JSON</i>	Низкая	Полная (встроено)
<i>SOAP</i>	<i>HTTP, SMTP</i>	<i>XML (WSDL)</i>	Высокая	Только сторонние пакеты
<i>GraphQL</i>	<i>HTTP</i>	<i>JSON (схема SDL)</i>	Средняя	Сторонние (<i>graphene</i>)
<i>gRPC</i>	<i>HTTP/2</i>	<i>Protocol Buffers</i> (бинарный)	Высокая	Сторонние (<i>grpcio</i>)
<i>Web-Socket</i>	<i>TCP</i>	Произвольный	Средняя	<i>Channels</i> (внешний пакет)

Для разрабатываемого программного продукта *VoltStream* в качестве архитектурного стиля был осознанно выбран *REST* (*Representational State Transfer*). Данное решение обусловлено целым рядом весомых технических причин, главная из которых заключается в том, что сложная инженерная топология схем исключительно хорошо поддается декомпозиции и описывается через набор независимых информационных ресурсов (схема, ревизия, папка, трансформатор, линия, тип потребителя), которые логично, прозрачно и естественно отображаются в иерархической структуре унифицированных идентификаторов (*URI*).

Все необходимые операции по управлению и манипуляции этими данными идеально и без дополнительных усложнений укладываются в классическую семантику стандартных методов протокола *HTTP*, что обеспечивает предсказуемость сетевого взаимодействия.

Немаловажную роль сыграл и выбор легковесного формата обмена данными *JSON*, который не только изначально и нативно совместим с используемым *JavaScript*-клиентом на стороне браузера, но и отличается значительно большей

компактностью по сравнению с громоздким *XML*, что положительно сказывается на пропускной способности.

Полноценная поддержка концепции *REST* в серверном стеке на базе *Django* реализуется штатными и проверенными средствами самого фреймворка, что позволяет сохранить чистоту кодовой базы и высокую производительность системы без необходимости интеграции и последующей поддержки тяжеловесных сторонних библиотек.

На рисунке 1.1 представлена наглядная общая схема этой тщательно спроектированной масштабируемой клиент-серверной архитектуры с интегрированным *REST-API* детально.

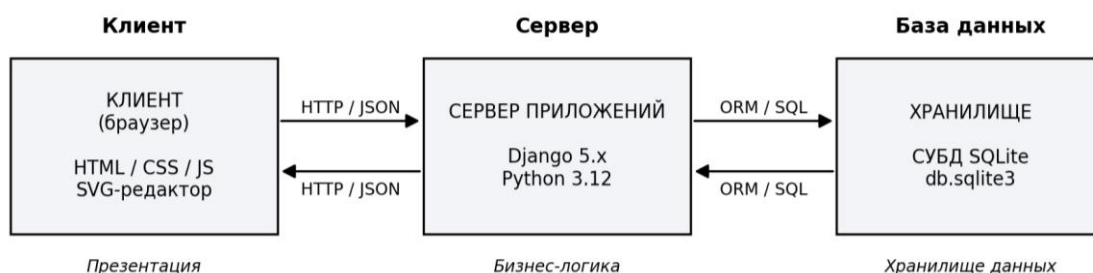


Рисунок 1.1 – Клиент-серверная архитектура *VoltStream* с *REST-API*

В качестве уточнения базовой архитектуры применён шаблон *Model-View-Template (MVT)*, реализованный в фреймворке *Django* [6].

Шаблон *MVT* является адаптацией классического *Model-View-Controller (MVC)* к веб-приложениям и предусматривает строгое разделение ответственности: модели описывают предметную область и доступ к данным, шаблоны отвечают за *HTML*-представление, а функции-обработчики (*views*) выполняют роль контроллера, принимая *HTTP*-запросы клиента и формируя *REST*-ответы.

1.3 Программные средства для реализации программного продукта

Для реализации программного продукта необходимо обоснованно выбрать язык программирования, веб-фреймворк, систему управления базами данных, технологии клиентской части и интегрированную среду разработки.

В таблице 1.3 приведено сравнение языков программирования по следующим критериям: наличие зрелого веб-фреймворка, скорость разработки, пригодность для построения веб-приложений, распространённость в отрасли и уровень поддержки сообщества.

Таблица 1.3 – Сравнение языков программирования

Язык	Веб-фреймворк	Скорость разработки	Поддержка сообщества
<i>Python</i>	<i>Django, Flask, FastAPI</i>	Очень высокая	Очень высокая
<i>JavaScript (Node.js)</i>	<i>Express, NestJS</i>	Высокая	Очень высокая
<i>Java</i>	<i>Spring Framework</i>	Средняя	Высокая
<i>C# (.NET)</i>	<i>ASP.NET Core</i>	Средняя	Высокая
<i>PHP</i>	<i>Laravel, Symfony</i>	Высокая	Средняя

В качестве языка программирования серверной части выбран *Python* версии 3.12 [7]. *Python* обладает лаконичным синтаксисом, развитой стандартной библиотекой и зрелым набором веб-фреймворков, что позволяет в сжатые сроки реализовать готовое решение. Кроме того, язык имеет встроенную поддержку работы с реляционными базами данных через *ORM*-библиотеки и широко применяется в инженерных расчётных задачах, что упрощает дальнейшее развитие проекта в направлении автоматизации расчётов режимов сети.

В таблице 1.4 представлено сравнение веб-фреймворков на языке *Python*.

Таблица 1.4 – Сравнение веб-фреймворков для языка *Python*

Фреймворк	<i>ORM</i> в составе	Админ-панель	Скорость старта проекта
<i>Django</i>	Да (встроенный)	Да (встроена)	Очень высокая
<i>Flask</i>	Нет (внешний <i>SQLAlchemy</i>)	Нет	Высокая
<i>FastAPI</i>	Нет (внешний)	Нет	Высокая
<i>Pyramid</i>	Нет (внешний)	Нет	Средняя

Выбран фреймворк *Django*, поскольку в его составе уже присутствуют *URL*-маршрутизатор, *ORM*, шаблонизатор, встроенная система аутентификации и развитая админ-панель. Маршрутизатор *Django* поддерживает разбор параметров пути (*path converters*) и привязку обработчика-функции к каждому *HTTP*-методу, что позволяет реализовать *REST-API* без подключения внешних библиотек. Класс *JsonResponse* и модуль *json* стандартной библиотеки *Python* обеспечивают сериализацию топологии схемы и ответов справочников в формат *JSON*. Это позволяет сосредоточиться на бизнес-логике приложения, не разрабатывая указанные подсистемы с нуля. Для серверной части используется *Django* версии 5.x с шаблоном *MVT*.

В таблице 1.5 приведено сравнение систем управления базами данных.

Таблица 1.5 – Сравнение СУБД

СУБД	Тип	Размещение	Сложность администрирования	Применимость для прототипа
<i>SQLite</i>	Реляционная	Встроенная (файл)	Минимальная	Высокая
<i>PostgreSQL</i>	Реляционная	Серверная	Средняя	Высокая
<i>MySQL</i>	Реляционная	Серверная	Средняя	Высокая
<i>MongoDB</i>	Документная	Серверная	Высокая	Низкая

В качестве СУБД для текущего этапа разработки выбрана *SQLite* [8] – встроенная реляционная база данных, не требующая отдельного сервера и поставляемая как одиночный файл. Этого достаточно для прототипа и тестирования. При этом *ORM Django* полностью абстрагирует обращения к БД, что в дальнейшем позволит без изменения кода переключиться на *PostgreSQL* или *MySQL*.

В качестве клиентской технологии используется чистый *JavaScript* (*ECMAScript 2020*) без использования сборщиков и фреймворков (*React*, *Vue*, *Angular*). Это решение принято в связи с тем, что графический редактор схем построен на технологии *SVG* и не требует сложного управления состоянием в стиле компонентного подхода. Применение фреймворка добавило бы избыточную сложность сборки и увеличило размер клиентской части без явного выигрыша в качестве пользовательского интерфейса. Для оформления административной панели подключена CSS-библиотека *Tailwind CSS* [9].

В таблице 1.6 приведено сравнение *IDE*, на основании которого в качестве интегрированной среды разработки выбран *JetBrains PyCharm Professional* [10].

Таблица 1.6 – Сравнение интегрированных сред разработки

<i>IDE</i>	Поддержка <i>Django</i>	Отладчик	Интеграция с СУБД
<i>JetBrains PyCharm</i>	Глубокая (нативная)	Полнофункциональный	Встроенная
<i>Visual Studio Code</i>	Через расширения	Хороший	Через расширения
<i>Sublime Text</i>	Через плагины	Ограниченный	Через плагины
<i>Vim/Neovim</i>	Через плагины	Ограниченный	Через плагины

PyCharm обеспечивает максимально глубокую интеграцию с веб-фреймворком *Django*, поддержку автодополнения шаблонов, встроенный отладчик и

удобный инструмент просмотра *SQLite*-файла, что в совокупности существенно ускоряет процесс разработки и значительно упрощает локализацию возникающих ошибок.

На основе анализа существующих решений были четко определены технические требования, согласно которым разрабатываемый продукт реализуется как масштабируемое клиент-серверное приложение для совместной многопользовательской работы.

Взаимодействие участвующих сторон организуется в строгом соответствии со стилем *REST* через сетевой протокол *HTTP* и текстовый формат *JSON*, где сложная топология схемы сериализуется в *JSON*-структуру и надежно сохраняется в реляционной БД с полноценной поддержкой версионирования.

Интерактивный клиентский *SVG*-редактор безопасно формирует запросы к целевым эндпоинтам сервера без прямого физического доступа к базе данных, а для удобного централизованного управления системными справочниками и учетными записями пользователей предусматривается отдельная административная панель с классической реализацией *REST-CRUD*.

2 ПРОЕКТИРОВАНИЕ КЛИЕНТ-СЕРВЕРНОГО ПРИЛОЖЕНИЯ И *REST-API*

2.1 Архитектура клиент-серверного приложения

Программный продукт *VoltStream* реализован в виде клиент-серверного веб-приложения с *REST-API* на серверной стороне и одностраничным *JavaScript*-клиентом на стороне браузера. Шаблон проектирования серверной части – *MVT*, реализованный в фреймворке *Django*. Общая схема архитектуры с указанием слоёв и направлений *HTTP*-обмена приведена на рисунке 2.1.

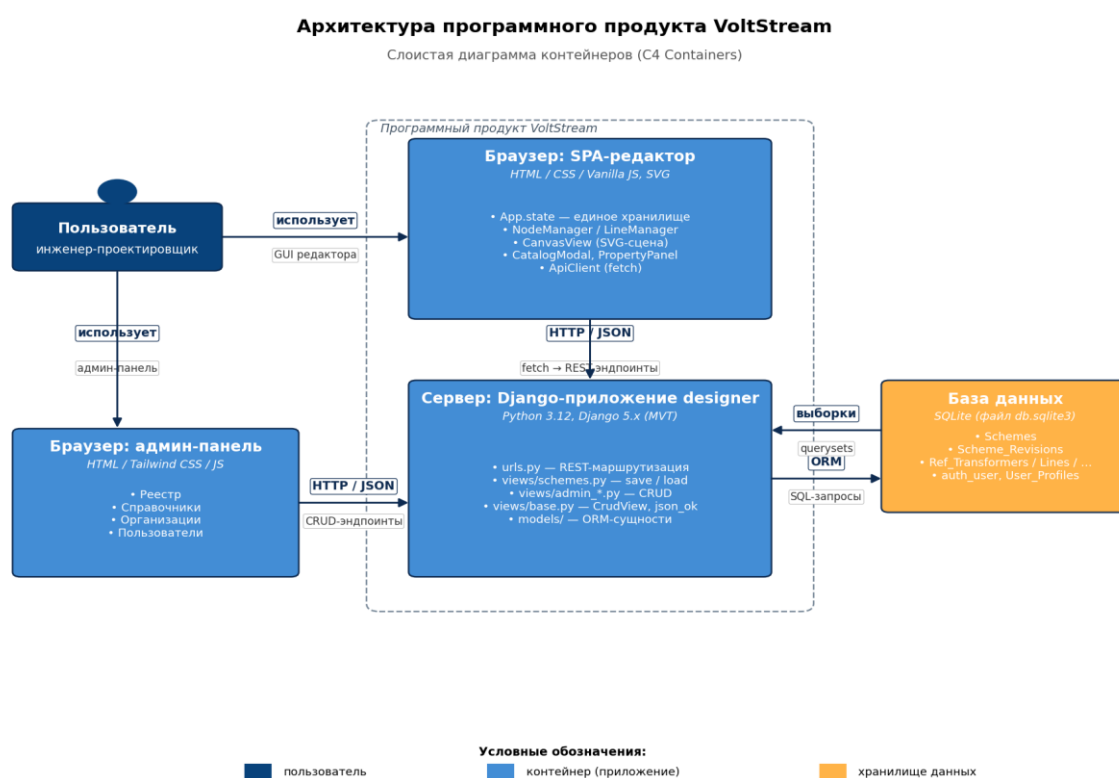


Рисунок 2.1 – Архитектура программного продукта *VoltStream*

Клиентская часть выполняется в браузере пользователя и состоит из *HTML*-разметки, *CSS*-стилей и набора модулей на чистом *JavaScript*. Все модули регистрируют свои объекты в едином пространстве *App*, реализуя модульную архитектуру без сборщика [11]. Графическое представление однолинейных схем построено на *SVG*, что позволяет масштабировать изображение без потери качества и программно манипулировать его элементами через *DOM-API*. Связь с сервером возложена на отдельный модуль *ApiClient*, инкапсулирующий формирование заголовков *HTTP*, передачу *JSON*-тела запроса и разбор *JSON*-ответа.

Серверная часть реализована на *Django* и состоит из приложения *designer*, в котором сосредоточена вся бизнес-логика.

Серверная часть отвечает за приём *HTTP*-запросов от клиента, валидацию входного *JSON*, обращение к базе данных через *ORM*, выполнение бизнес-операций и формирование *JSON*-ответа с соответствующим *HTTP*-статусом.

Прямого доступа клиента к базе данных не предусмотрено: вся работа со схемой и справочниками ведётся исключительно через *REST-API*.

2.2 Проектирование *REST-API*

Состав *REST-API* спроектирован исходя из принципа отображения ресурсов предметной области в *URI* и закрепления за ресурсами семантики стандартных методов *HTTP*. Выделены следующие группы ресурсов: схемы и их ревизии, реестр папок, каталог справочников, административные ресурсы (трансформаторы, ЛЭП, типы потребителей, организации, пользователи).

В таблице 2.1 приведены перечень эндпоинтов *API* и закреплённые за ними *HTTP*-методы.

Таблица 2.1 – Состав *REST-API* программного продукта *VoltStream*

<i>URI</i>	Метод	Назначение
<i>1</i>	<i>2</i>	<i>3</i>
/	<i>GET</i>	Главная страница редактора схем (<i>HTML</i>)
/admin-panel/	<i>GET</i>	Административная панель (<i>HTML</i>)
/api/save/	<i>POST</i>	Сохранение новой ревизии схемы (<i>JSON</i>)
/api/list/	<i>GET</i>	Список ревизий схем (<i>JSON</i>)
/api/load/<rev_id>/	<i>GET</i>	Чтение топологии конкретной ревизии
/api/registry/	<i>GET</i>	Реестр папок и схем (<i>JSON</i> -дерево)
/get_catalog_nodes/	<i>GET</i>	Каталог справочника с пагинацией
/api/get_node_details/	<i>GET</i>	Свойства одного объекта справочника
/api/admin/transformers/	<i>GET/POST/PUT/DELETE</i>	<i>CRUD</i> справочника трансформаторов
/api/admin/lines/	<i>GET/POST/PUT/DELETE</i>	<i>CRUD</i> справочника ЛЭП

Продолжение таблицы 2.1

1	2	3
<i>/api/admin/consumers/</i>	<i>GET/POST/PUT/DELETE</i>	<i>CRUD</i> справочника потребителей
<i>/api/admin/folders/</i>	<i>POST/PUT/DELETE</i>	Управление папками реестра

В таблице 2.2 приведено соответствие операций над ресурсами и методов *HTTP*, использованное в проекте; оно отвечает классическим *REST*-договорённостям: *GET* – безопасное и идиempotentное чтение, *POST* – создание нового ресурса, *PUT* – идиempotentное обновление существующего, *DELETE* – удаление.

Таблица 2.2 – Семантика методов *HTTP* в *REST-API VoltStream*

Метод	Назначение	Безопасный	Идиempotentный
<i>GET</i>	Чтение представления ресурса	Да	Да
<i>POST</i>	Создание нового ресурса	Нет	Нет
<i>PUT</i>	Полное обновление ресурса	Нет	Да
<i>DELETE</i>	Удаление ресурса	Нет	Да

Коды состояний *HTTP*, возвращаемые сервером, унифицированы по всем эндпоинтам: 200 *OK* – успешное выполнение, 302 *Found* – редирект на страницу авторизации *Django*, 400 *Bad Request* – некорректный *JSON* во входном запросе или нарушение бизнес-правил, 403 *Forbidden* – недостаточно прав, 404 *Not Found* – запрошенный ресурс отсутствует. Тело ответа во всех случаях, кроме редиректа, содержит *JSON* с результатом операции либо описанием ошибки в поле *error*.

На рисунке 2.2 показан пример формата *JSON*, передаваемого клиентом серверу при сохранении схемы (*POST /api/save/*).

```
{
  "name": "Подстанция №14",
  "folder_id": 7,
  "nodes": [
    {
      "id": "n1", "type": "ТП", "x": 120, "y": 80,
      "db_id": 12, "label": "ТМ-400/10"
    },
    {
      "id": "n2", "type": "Опора", "x": 260, "y": 80
    }
  ],
  "lines": [
    {
      "from": "n1", "to": "n2", "db_id": 314,
      "mark": "СИП-2 4x25"
    }
  ]
}
```

Рисунок 2.2 – Формат *JSON*-сообщения *POST /api/save/*

Слой хранения данных представлен реляционной СУБД *SQLite*. Все обращения к БД выполняются через *Django ORM*. Файл базы данных *db.sqlite3* размещён в корне проекта; полученная при разборе *JSON*-топология сохраняется в поле *topology_data* таблицы *Scheme_Revisions*.

2.3 Диаграмма классов приложения

Диаграмма классов разрабатываемого приложения отражает основные сущности предметной области и связи между ними. В системе выделены три группы классов:

- ядро системы – *Organization*, *Role*, *UserProfile*, *AuditLog*;
- схемы – *Folder*, *Scheme*, *SchemeRevision*;
- справочники – *RefTransformer*, *RefLine*, *RefConsumerType*.

Общая диаграмма классов приведена на рисунке 2.3. На диаграмме показаны имена классов, их основные атрибуты и связи между ними с указанием кратности.

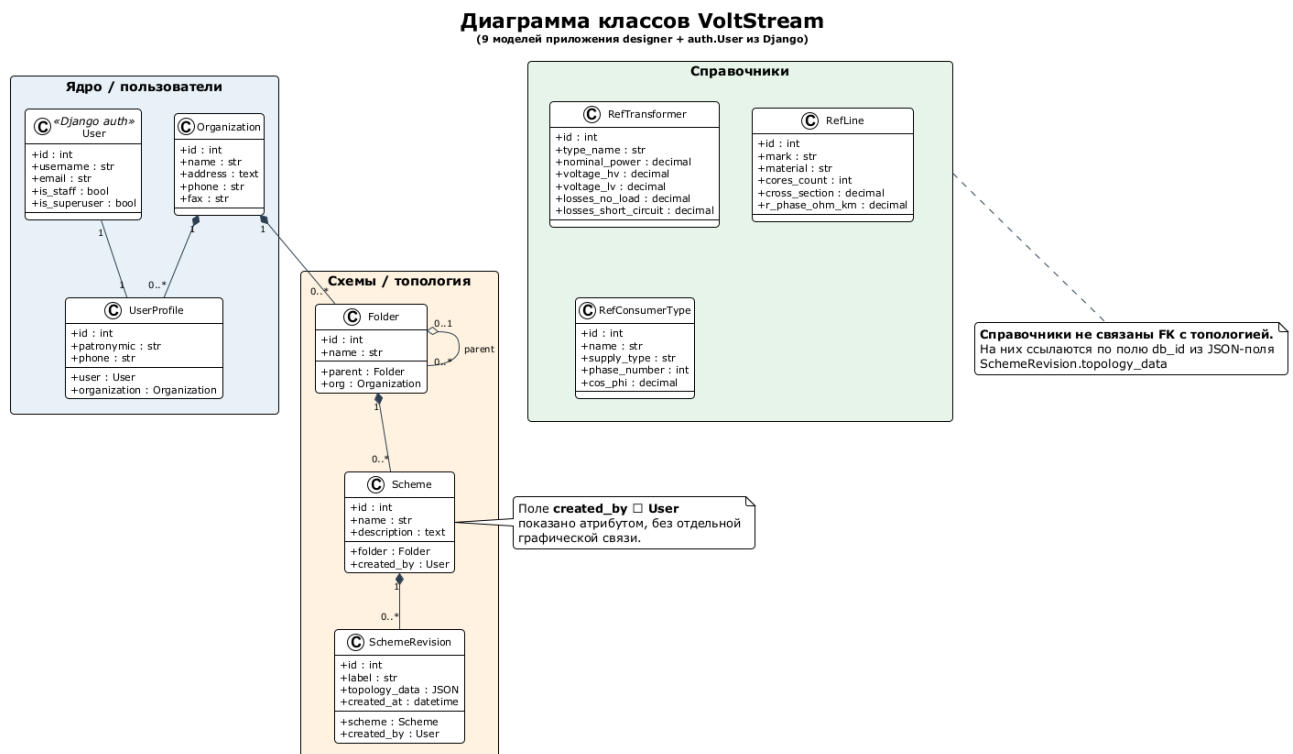


Рисунок 2.3 – Диаграмма классов программного продукта *VoltStream*

Класс *Organization* представляет юридическое лицо, в рамках которого ведётся учёт схем сетей. Класс *UserProfile* (связь 1:1 с базовым классом *User* фреймворка *Django*) расширяет стандартный пользовательский профиль полями отчества, телефона и ссылкой на организацию. Класс *Folder* образует

иерархическое дерево папок (самоссылка через поле *parent*), в которых хранятся схемы (*Scheme*). Каждой схеме соответствует произвольное число ревизий (*SchemeRevision*), хранящих топологию в *JSON*-формате. Справочные классы *RefTransformer*, *RefLine* и *RefConsumerType* содержат паспортные данные элементов сети, используемых при построении схем.

2.4 Диаграмма последовательности обмена по *REST-API*

Для отражения динамического взаимодействия между клиентом и сервером по *REST-API* в работе приведена диаграмма последовательности для типового сценария «сохранение схемы пользователем». Диаграмма приведена на рисунке 2.4.

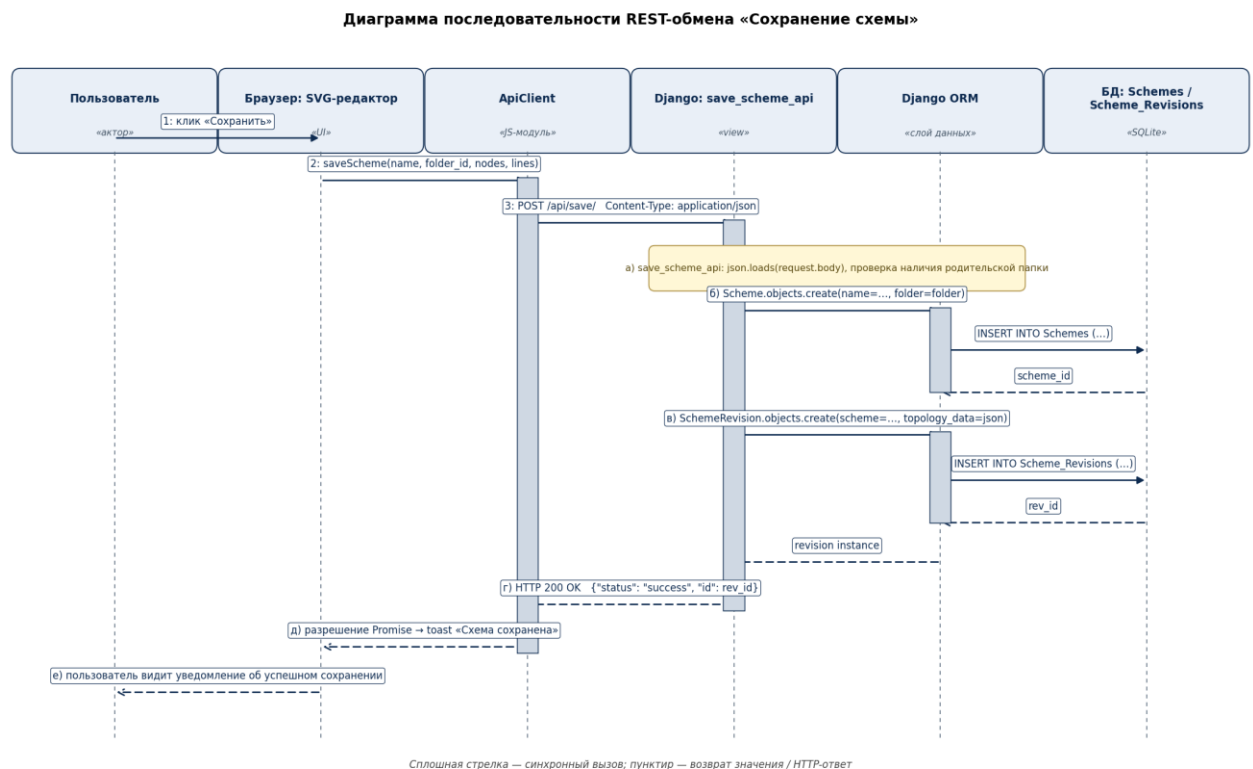


Рисунок 2.4 – Диаграмма последовательности *REST*-обмена «Сохранение схемы»

На диаграмме 2.4 видно, что пользователь, нажав кнопку «Сохранить» в редакторе, инициирует следующую последовательность действий:

а) клиентский модуль *ApiClient* формирует *POST*-запрос к эндпоинту */api/save/* с заголовком *Content-Type: application/json*, передавая в теле запроса имя схемы, идентификатор папки и *JSON*-описание топологии;

б) серверный обработчик *save_scheme_api* выполняет парсинг тела запроса методом *json.loads* и проверку наличия родительской папки;

- в) при необходимости создаётся новая запись в таблице *Schemes* через *Django ORM*;
- г) в таблицу *Scheme_Revisions* добавляется новая запись с *JSON*-снимком топологии и текущим временем;
- д) клиенту возвращается ответ *HTTP 200 OK* с *JSON*-телом, содержащим идентификатор созданной ревизии;
- е) клиентский модуль обновляет реестр и выводит уведомление об успешном сохранении.

2.5 Проектирование модели данных для хранения передаваемой топологии

Несмотря на то, что центральным элементом разрабатываемого продукта является *REST-API*, корректная работа сервера невозможна без проектирования модели данных, в которой принятая от клиента топология будет сохранена и из которой эта же топология впоследствии будет извлечена для возврата по запросу *GET /api/load/<id>/.* Проектирование выполнено в два этапа. На первом этапе построена логическая модель – выделены сущности предметной области, определены атрибуты и связи между сущностями (*ER*-диаграмма). На втором этапе логическая модель преобразована в физическую модель – конкретные таблицы реляционной БД с указанием типов данных столбцов, ограничений целостности и индексов. При проектировании соблюдены требования третьей нормальной формы.

В предметной области выделены следующие сущности: организация, пользователь (профиль), роль, папка, схема, ревизия схемы, трансформатор, линия электропередачи, тип потребителя, запись журнала аудита. Между сущностями установлены связи, представленные на *ER*-диаграмме (рисунок 2.5).

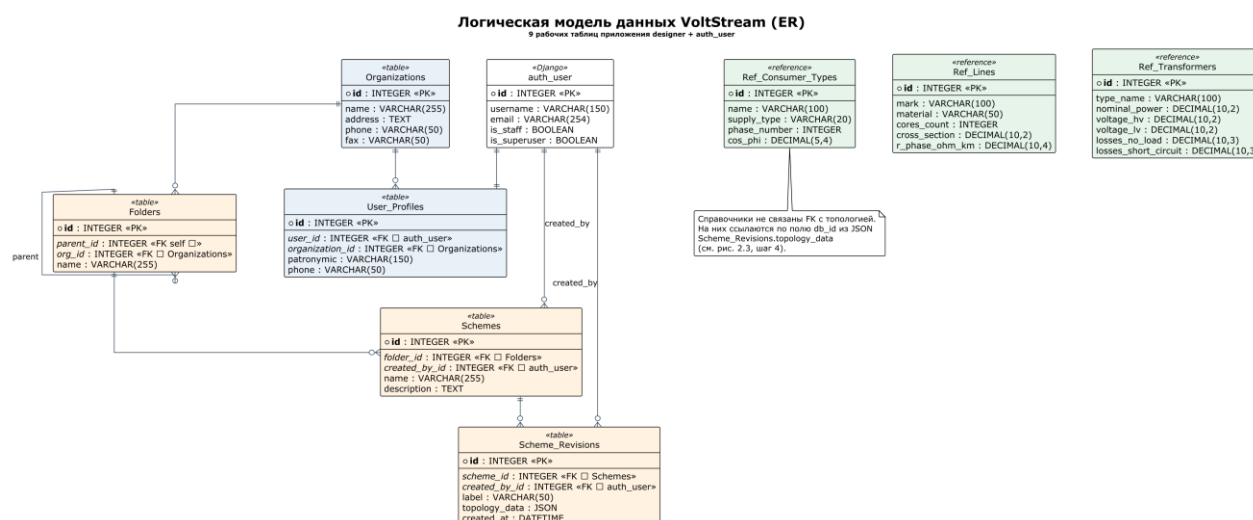


Рисунок 2.5 – Логическая модель данных (*ER*-диаграмма)

Перечень связей между сущностями приведён ниже.

- *Organization* → *UserProfile* (1 : N): Одна организация может иметь много пользователей;
- *Organization* → *Folder* (1 : N): Одна организация может иметь много корневых папок;
- *Folder* → *Folder* (1 : N): Папка может содержать вложенные папки (самоссылка);
- *Folder* → *Scheme* (1 : N): Одна папка содержит несколько схем;
- *Scheme* → *SchemeRevision* (1 : N): Одна схема имеет несколько ревизий;
- *User* → *SchemeRevision* (1 : N): Один пользователь может создать несколько ревизий;
- *Role* → *UserProfile* (1 : N): Одной роли может соответствовать несколько профилей.

В таблице 2.4 приведён перечень 12 таблиц физической модели данных *VoltStream*; имена таблиц назначены явно через параметр *db_table* моделей *Django*.

Таблица 2.4 – Физические таблицы базы данных *VoltStream*

Имя таблицы	Сущность	Назначение
<i>Organizations</i>	<i>Organization</i>	Юридические лица – владельцы папок и схем
<i>User_Profiles</i>	<i>UserProfile</i>	Расширение <i>auth_user</i> (отчество, телефон, организация)
<i>Roles</i>	<i>Role</i>	Справочник ролей пользователей
<i>Folders</i>	<i>Folder</i>	Иерархическое дерево папок реестра
<i>Schemes</i>	<i>Scheme</i>	Именованные однолинейные схемы
<i>Scheme_Revisions</i>	<i>SchemeRevision</i>	Снимки схем (JSON-топология)
<i>Ref_Transformers</i>	<i>RefTransformer</i>	Справочник типов трансформаторов
<i>Ref_Lines</i>	<i>RefLine</i>	Справочник марок проводов и кабелей
<i>Ref_Consumer_Types</i>	<i>RefConsumerType</i>	Справочник типов потребителей
<i>Audit_Logs</i>	<i>AuditLog</i>	Журнал действий пользователей
<i>auth_user</i>	–	Стандартная таблица пользователей <i>Django</i>
<i>jango_session</i>	–	Стандартная таблица сессий <i>Django</i>

В таблице 2.5 приведена структура ключевой таблицы *Scheme_Revisions*, отвечающей за хранение JSON-топологии, переданной по *REST-API*.

Таблица 2.5 – Структура таблицы *Scheme_Revisions*

Поле	Тип	Ограничения	Назначение
<i>id</i>	<i>INTEGER</i>	<i>PK, AUTOINCREMENT</i>	Идентификатор ревизии
<i>scheme_id</i>	<i>INTEGER</i>	<i>FK Schemes.id, NOT NULL</i>	Принадлежность схеме
<i>label</i>	<i>VAR-CHAR(50)</i>	<i>NOT NULL</i>	Метка версии (дата + время)
<i>topology_data</i>	<i>JSON</i>	<i>NOT NULL</i>	Топология схемы в JSON-формате
<i>created_at</i>	<i>DATETIME</i>	<i>NOT NULL, DEFAULT CURRENT_TIMESTAMP</i>	Дата создания ревизии
<i>created_by_id</i>	<i>INTEGER</i>	<i>FK auth_user.id, NULL</i>	Автор ревизии

В таблице 2.6 приведена структура справочной таблицы *Ref_Transformers*.

Таблица 2.6 – Структура таблицы *Ref_Transformers*

Поле	Тип	Назначение
<i>id</i>	<i>INTEGER PK</i>	Идентификатор записи
<i>type_name</i>	<i>VARCHAR(100)</i>	Тип трансформатора (марка)
<i>nominal_power</i>	<i>DECIMAL(10,2)</i>	Номинальная мощность, кВА
<i>losses_no_load</i>	<i>DECIMAL(10,3)</i>	Потери холостого хода, кВт
<i>losses_short_circuit</i>	<i>DECIMAL(10,3)</i>	Потери короткого замыкания, кВт
<i>voltage_hv</i>	<i>DECIMAL(10,2)</i>	Номинальное напряжение ВН, кВ
<i>voltage_lv</i>	<i>DECIMAL(10,2)</i>	Номинальное напряжение НН, кВ
<i>voltage_short_circuit_pct</i>	<i>DECIMAL(5,2)</i>	Напряжение короткого замыкания, %
<i>pbv_stages_count</i>	<i>INTEGER</i>	Количество ступеней ПБВ
<i>pbv_step_pct</i>	<i>DECIMAL(5,2)</i>	Шаг одной ступени ПБВ, %

Соблюдение третьей нормальной формы достигнуто следующими проектными решениями:

– 1НФ: все атрибуты атомарны – отсутствуют поля-списки и поля, содержащие несколько значений (исключение составляет поле *topology_data*, для хранения которого использован тип *JSON*, что является допустимой денормализацией для топологии переменной структуры);

– 2НФ: каждое неключевое поле полностью зависит от первичного ключа своей таблицы – справочные сведения вынесены в отдельные таблицы *Ref_Transformers*, *Ref_Lines*, *Ref_Consumer_Types*;

– 3НФ: устранены транзитивные зависимости – данные пользователя хранятся в *auth_user*, а пользовательский профиль и связь с организацией – в отдельной таблице *User_Profiles*, что исключает дублирование сведений об организации в каждой записи о пользователе.

Заполнение справочников выполнено реальными паспортными данными оборудования. По состоянию на момент тестирования в базе данных хранятся 78 типов трансформаторов, 6632 марки проводов и кабелей и 4 типа потребителей. Папок реестра – 10, схем – 3, ревизий – 4. Это подтверждает практическую применимость спроектированной модели.

2.6 Проектирование клиентского интерфейса

Пользовательский интерфейс разрабатываемого приложения *VoltStream* концептуально разделен на две изолированные функциональные области, ориентированные на различные сценарии использования: интерактивная рабочая область инженера-проектировщика схем и закрытая административная панель управления. Для каждой из этих предметных подобластей спроектирована собственная независимая *HTML*-страница, асинхронно обращающаяся к серверному ядру исключительно посредством спецификаций *REST-API*.

Главная страница визуального редактора схем (рисунок 2.6) скомпонована и состоит из четырех ключевых функциональных зон. Верхняя навигационная панель инструментов содержит элементы управления проектом, включая кнопки сохранения, загрузки и работы с ревизиями. Левая боковая панель представляет собой структурированную библиотеку компонентов, откуда типовые элементы сети могут переноситься на рабочее пространство. Основную площадь занимает масштабируемая центральная *SVG*-сцена, выступающая интерактивным холстом для визуального построения и трассировки схемы. Справа располагается динамическая панель свойств, контекстно отображающая редактируемые атрибуты выбранного в данный момент элемента. Для стандартизации сетевого обмена и контроля авторизации все исходящие обращения фронтенда к серверу инкапсулированы и проходят строго через единый модуль *ApiClient*.

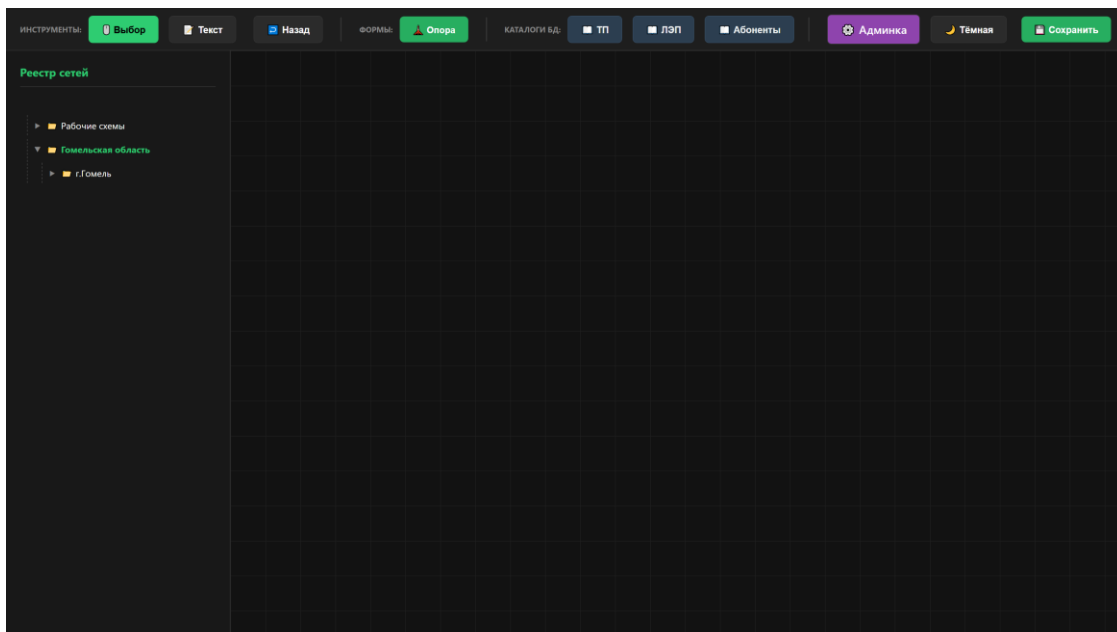


Рисунок 2.6 – Главная страница редактора схем

Административная панель (рисунок 2.7) реализована в виде одностраничного приложения с навигацией через меню по вкладкам: «Реестр», «Справочники», «Организации», «Пользователи». Каждая вкладка обращается к соответствующим *REST*-эндпоинтам и поддерживает поиск, сортировку, добавление, изменение и удаление записей.

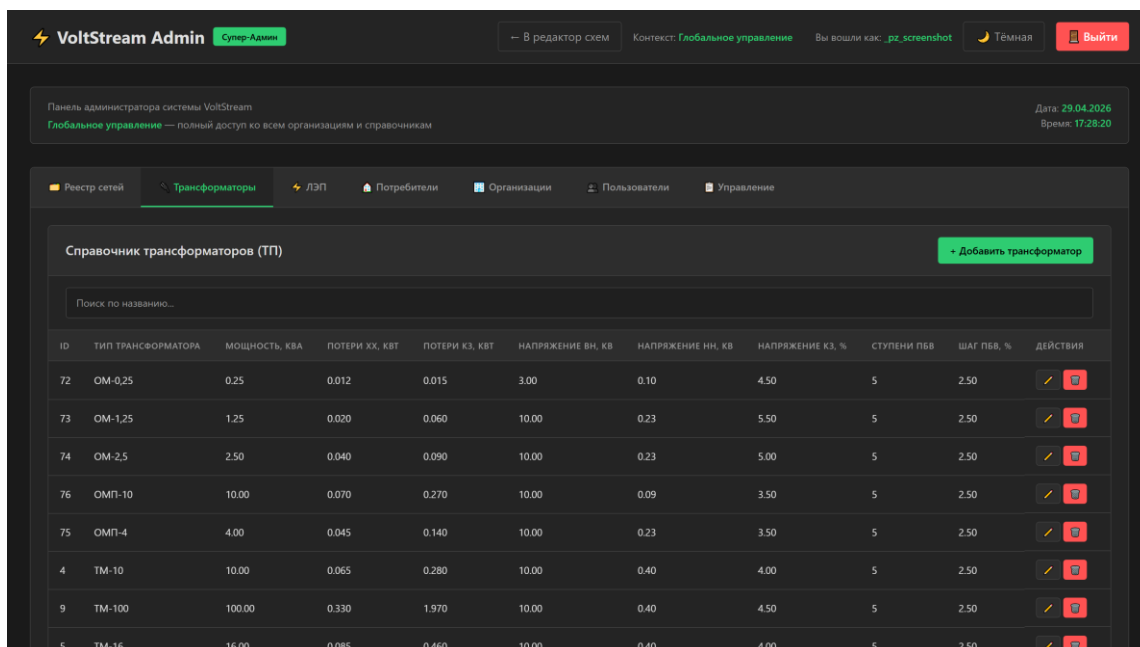


Рисунок 2.7 – Административная панель программы

Для выбора элементов справочника при размещении на схеме реализовано переиспользуемое модальное окно каталога (рисунок 2.8), поддерживающее

поиск по подстроке и пагинацию по 50 записей на страницу через *GET*-запрос к */get_catalog_nodes/*.

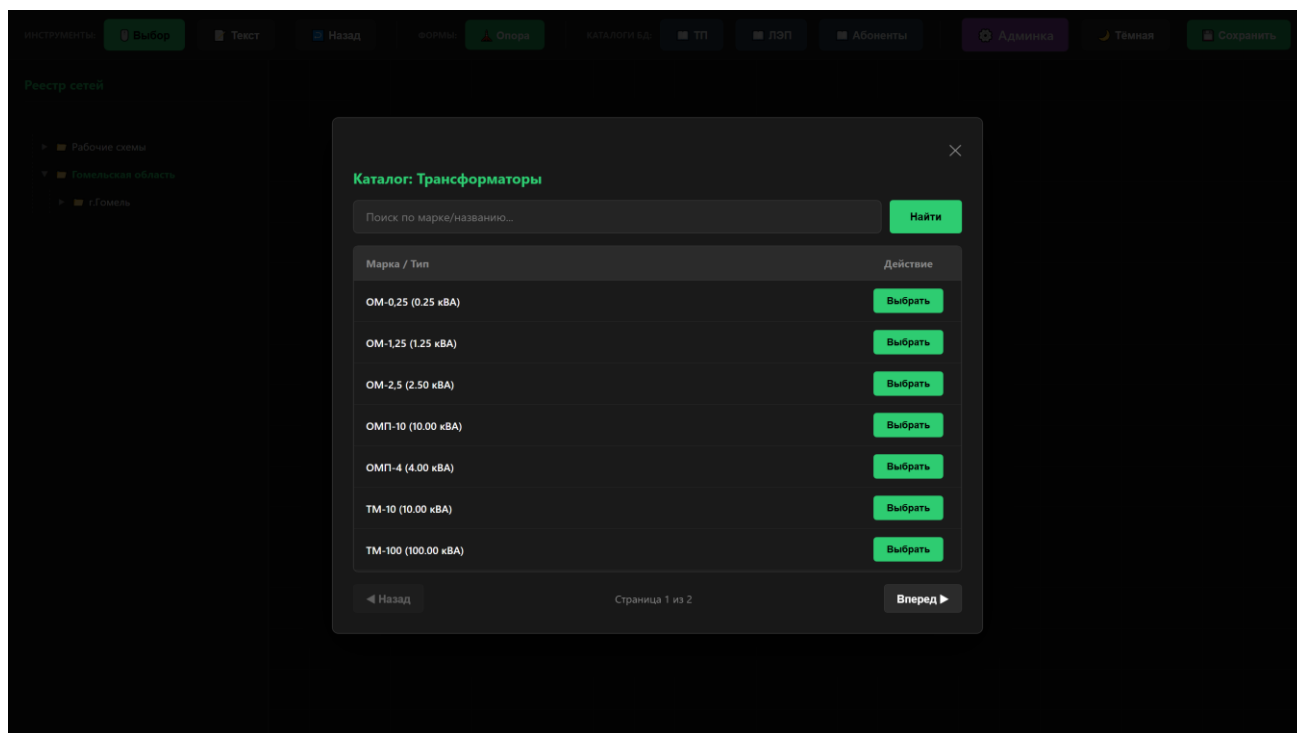


Рисунок 2.8 – Модальное окно каталога справочника

Выбор конкретной позиции в каталоге инициирует автоматическую передачу технических характеристик элемента в модель данных текущей схемы. Такое решение позволяет существенно ускорить процесс проектирования за счёт исключения ручного ввода параметров и обеспечения строгого соответствия проектных решений актуальной справочной информации.

3 РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ *REST-API*

3.1 Особенности реализации серверной части и *REST-API*

Программный продукт реализован на языке *Python* 3.12 с использованием веб-фреймворка *Django*. Серверная часть обрабатывает *HTTP*-запросы клиента и формирует *JSON*-ответы; общий объём исходного кода составляет около 1100 строк, разбитых на тематические пакеты *designer/models* и *designer/views*. Клиентская часть выполнена на чистом *JavaScript* и состоит из 8 модулей суммарным объёмом около 1500 строк, обращающихся к серверу исключительно через *fetch*-вызовы к *REST*-эндпоинтам. Структура файлов и каталогов проекта приведена в приложении Б (рисунок Б.1).

В качестве важной архитектурной особенности следует отметить ООП-слой для *CRUD*-эндпоинтов *REST-API* над справочниками. Базовый класс *CrudView* (*designer/views/base.py*) реализует методы *list*, *create*, *update*, *delete*, отображающиеся на *HTTP*-методы *GET*, *POST*, *PUT* и *DELETE* соответственно, с единой обработкой ошибок и проверкой прав доступа. Конкретные справочники описывают только декларативную схему полей. На рисунке 3.2 показан пример использования.

Пример декларативного описания справочника ЛЭП:

```
class LineCrud(CrudView):
    model = RefLine
    order_by = 'mark'
    fields_read = {
        'mark': 'mark',
        'material': 'material',
        'cores_count': 'cores_count',
        'cross_section': 'cross_section',
        'r_phase_ohm_km': 'r_phase_ohm_km',
    }
    fields_write = {
        'mark': ('mark', lambda v: v or ''),
        'material': ('material', lambda v: v or ''),
        'cores_count': ('cores_count', to_int),
        'cross_section': ('cross_section', to_float),
        'r_phase_ohm_km': ('r_phase_ohm_km', to_float),
    }
```

Рисунок 3.2 – Декларативное описание *CRUD* справочника ЛЭП

Клиентская часть построена на принципе единого источника истины: всё состояние редактора (узлы, линии, выбор, история, *viewport*) хранится в объекте

App.state, а все изменения проходят через единственную точку перерисовки *App.render()*. Менеджеры модулей (*NodeManager*, *LineManager*, *PropertyPanel*, *CatalogModal*, *CanvasView*, *ApiClient*) не дублируют состояние, а работают с общим хранилищем. Передача состояния на сервер выполняется только модулем *ApiClient* – единственной точкой, инкапсулирующей сетевой обмен. Это исключает рассинхронизацию представления, модели данных и серверного хранилища.

Особое внимание уделено алгоритму ортогональной отрисовки линий электропередачи. При перемещении узлов схемы любая ломаная линия, соединяющая два узла, автоматически выпрямляется в последовательность горизонтальных и вертикальных сегментов с прямыми углами. Это соответствует принятым в инженерной практике требованиям к оформлению однолинейных схем.

3.2 Функциональное тестирование *REST-API*

Функциональное тестирование разработанного *REST-API* выполнено методом «чёрного ящика» с использованием утилиты *curl* и встроенного отладочного веб-сервера *Django*, запущенного по адресу *http://127.0.0.1:8765/*. Утилита *curl* формирует *HTTP*-запросы с заданным методом, заголовком *Content-Type: application/json* и произвольным *JSON*-телом, что позволяет проверить *REST-API* независимо от клиентского веб-интерфейса. Тестирование проводилось в апреле 2026 г. Цель тестирования – подтвердить, что функциональные требования, сформулированные в разделе 1.3, корректно реализованы всеми эндпоинтами *API*.

В состав функционального тестирования включены:

- позитивные тесты основных пользовательских сценариев (сохранение схемы методом *POST*, загрузка ревизии методом *GET*, чтение каталога);
- тесты пагинации и поиска в каталоге справочников;
- тесты разграничения доступа (защита административных эндпоинтов);
- тесты обработки некорректных входных данных и возврата ожидаемых *HTTP*-кодов состояний.

Перечень тест-кейсов и фактически полученные результаты приведены в приложении В (таблица В.1).

Все 12 тест-кейсов завершились с ожидаемыми результатами и корректными *HTTP*-кодами состояний. Среднее время отклика серверных *REST*-эндпоинтов составило 6 миллисекунд, максимальное – 19 миллисекунд (отрисовка главной страницы). Это соответствует требованиям к быстродействию для веб-приложения, обслуживающего до 100 одновременных пользователей. Внешний вид ответа сервера на тестовое сохранение приведён в приложении В (рисунок В.1).

Тест 11 подтверждает корректную работу декоратора *staff_required*: при отсутствии авторизации сервер возвращает *HTTP 302 Found* с редиректом на

страницу входа *Django*, что предотвращает несанкционированный доступ к административному *REST-API*.

3.3 Unit-тесты

Помимо ручного функционального тестирования через *curl* разработан комплект автоматических модульных тестов *REST-API* на базе встроенной системы тестирования *Django* (*django.test.TestCase* и *Client*). Класс *Client* эмулирует *HTTP*-клиента и позволяет проверять *REST*-эндпоинты без поднятия отдельного сервера.

Использование данного инструментария также обеспечивает автоматическое создание изолированной тестовой базы данных для каждого запуска, что гарантирует независимость проверок друг от друга и исключает влияние побочных эффектов.

Тесты размещены в файле *designer/tests.py* и запускаются командой *python manage.py test designer*.

Разработаны три тестовых класса покрывающие наиболее критичные участки бизнес-логики приложения:

- а) *SaveLoadSchemeTest* – проверяет полный цикл сохранения и загрузки схемы по *HTTP API*;
- б) *CatalogSearchPaginationTest* – проверяет пагинацию, поиск по подстроке и обработку некорректной категории;
- в) *TransformerCrudTest* – проверяет *CRUD*-эндпоинты справочника трансформаторов через ООП-слой *CrudView*.

Описание тестовых классов и проверяемые сценарии приведены в приложении В (таблица В.2).

В процессе тестирования осуществляется валидация не только успешного выполнения операций (с ожиданием кодов ответа *HTTP 200 OK* и *201 Created*), но и корректной реакции системы на ошибочные данные (генерация *HTTP 400 Bad Request*).

Запуск всего набора тестов выполняется командой *python manage.py test designer -v 2*. По итогам запуска в апреле 2026 г. все 7 тестов завершились со статусом *OK*, общее время выполнения – 1,486 с. Скриншот терминала с результатом запуска приведён в приложении В (рисунок В.2).

Высокая скорость выполнения позволяет в перспективе легко интегрировать данный набор проверок в пайплайн непрерывной интеграции (*CI/CD*) для автоматического контроля качества кода.

Согласно протоколу запуска все тесты завершились успешно, что подтверждает корректность реализации *REST-API* программного продукта. Фрагмент кода теста *SaveLoadSchemeTest* приведён на рисунке 3.5.

```

class SaveLoadSchemeTest(TestCase):
    def setUp(self):
        self.client = Client()
        self.org = Organization.objects.create(name='Тестовая')
        self.folder = Folder.objects.create(
            name='Тестовая папка', org=self.org)

    def test_save_then_load_scheme(self):
        topology = {'name': 'Тестовая схема',
                    'folder_id': self.folder.id,
                    'nodes': [...], 'lines': [...]}
        save = self.client.post('/api/save/',
                                data=json.dumps(topology),
                                content_type='application/json')
        self.assertEqual(save.status_code, 200)
        rev_id = save.json()['id']
        load = self.client.get(f'/api/load/{rev_id}/')
        self.assertEqual(load.status_code, 200)

```

Рисунок 3.5 – Фрагмент исходного кода *unit*-теста *SaveLoadSchemeTest*

Совокупный результат функционального и модульного тестирования подтверждает, что разработанный *REST-API* программного продукта *VoltStream* корректно реализует клиент-серверную передачу топологии электрической сети, удовлетворяет требованиям технического задания и пригоден к опытной эксплуатации.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы на тему «Программный продукт для клиент-серверной передачи топологии электрических сетей через *REST-API* на базе *Django*» разработан и протестирован программный продукт *VoltStream*, обеспечивающий построение однолинейных схем распределительных сетей 0,4 кВ в браузере и передачу топологии между клиентом и сервером по *REST*-протоколу.

В рамках работы выполнены следующие задачи:

- проведён аналитический обзор существующих программных решений и архитектурных стилей клиент-серверного взаимодействия (*REST*, *SOAP*, *GraphQL*, *gRPC*, *WebSocket*), по результатам которого в качестве стиля интеграции выбран *REST* с форматом *JSON* и стандартными методами *HTTP*;
- обоснован выбор языка программирования *Python 3.12*, фреймворка *Django*, СУБД *SQLite*, чистого *JavaScript* для клиентской части и среды разработки *JetBrains PyCharm*;
- спроектировано *REST-API* из 12 групп эндпоинтов с закреплёнными за ними методами *HTTP*, форматом *JSON*-сообщений и кодами состояний; разработана модель данных в третьей нормальной форме (12 таблиц) для хранения принимаемой по *API* топологии;
- разработан программный продукт на базе *Django*, поддерживающий построение схем, управление справочниками, иерархию папок реестра, версионирование схем и разграничение прав доступа через *REST-API*;
- проведено функциональное тестирование *REST-API* (12 тест-кейсов с использованием *curl*) и модульное тестирование (7 *unit*-тестов на *django.test.Client*); все тесты пройдены успешно.

Полученное программное обеспечение может быть использовано проектными организациями для удалённой совместной работы со схемами распределительных сетей через единый *REST-API*. В качестве направлений дальнейшего развития могут быть рассмотрены: расчёт установившихся режимов, переход на серверную СУБД *PostgreSQL*, разработка альтернативных клиентов (мобильное приложение, командная утилита) поверх существующего *REST-API* и интеграция с геоинформационными системами.

Список использованных источников

1. AutoCAD Electrical : руководство пользователя [Электронный ресурс]. – Режим доступа : <https://www.autodesk.ru/products/autocad-electrical>. – Дата доступа : 10.04.2026.
2. RastrWin3 : официальный сайт [Электронный ресурс]. – Режим доступа : <http://www.rastrwin.ru>. – Дата доступа : 10.04.2026.
3. EnergyCS Электрика : описание программы [Электронный ресурс] / CSoft Development. – Режим доступа : <https://www.csoft.ru/products/energycs>. – Дата доступа : 10.04.2026.
4. DigSILENT PowerFactory : технический справочник [Электронный ресурс]. – Режим доступа : <https://www.digsilent.de>. – Дата доступа : 10.04.2026.
5. Van Steen, M. Distributed Systems / M. van Steen, A. S. Tanenbaum. – 4th ed. – Leiden : Maarten van Steen, 2023. – 648 p.
6. Django Documentation : official site [Electronic resource]. – Режим доступа : <https://docs.djangoproject.com/en/5.0/>. – Дата доступа : 12.04.2026.
7. Рамальо, Л. Python. К вершинам мастерства / Л. Рамальо ; пер. с англ. – 2-е изд. – СПб. : Питер, 2022. – 1088 с.
8. SQLite : the world's most used database [Electronic resource]. – Режим доступа : <https://www.sqlite.org>. – Дата доступа : 12.04.2026.
9. Tailwind CSS : framework documentation [Electronic resource]. – Режим доступа : <https://tailwindcss.com/docs>. – Дата доступа : 12.04.2026.
10. JetBrains PyCharm : документация [Электронный ресурс]. – Режим доступа : <https://www.jetbrains.com/help/pycharm/>. – Дата доступа : 12.04.2026.
11. Mozilla Developer Network. JavaScript : справочник [Электронный ресурс]. – Режим доступа : <https://developer.mozilla.org/ru/docs/Web/JavaScript>. – Дата доступа : 14.04.2026.

ПРИЛОЖЕНИЕ А

(справочное)

Структура проекта

```
Проект/
├── designer/
│   ├── management/
│   │   └── commands/
│   │       └── import_from_excel.py
│   ├── migrations/
│   │   ├── 0001_initial.py
│   │   ├── 0002_refpole.py
│   │   ├── 0003_refpole_branches_count.py
│   │   ├── 0004_alter_folder_org_alter_folder_parent_and_more.py
│   │   ├── 0005_remove_auditlog_user_alter_scheme_created_by_and_more.py
│   │   ├── 0006_remove_reftransformer_label_on_scheme_and_more.py
│   │   ├── 0007_delete_refpole_remove_refline_is_tap_to_single_pole_and_more.py
│   │   ├── 0008_remove_refconsumertype_label_on_scheme.py
│   │   ├── 0009_organization_fax_organization_phone_userprofile.py
│   │   └── __init__.py
│   ├── models/
│   │   ├── __init__.py
│   │   ├── calculations.py
│   │   ├── core.py
│   │   ├── references.py
│   │   └── schemes.py
│   ├── static/
│   │   ├── designer/
│   │   │   ├── css/
│   │   │   ├── icons/
│   │   │   ├── js/
│   │   │   └── index.html
│   │   └── templates/
│   │       ├── designer/
│   │       │   ├── css/
│   │       │   ├── js/
│   │       │   ├── admin_panel.html
│   │       │   ├── catalog_modal.html
│   │       │   └── index.html
│   │       └── views/
│   │           ├── __init__.py
│   │           ├── admin_consumers.py
│   │           ├── admin_lines.py
│   │           ├── admin_organizations.py
│   │           ├── admin_transformers.py
│   │           ├── admin_users.py
│   │           ├── base.py
│   │           ├── catalog.py
│   │           ├── pages.py
│   │           └── schemes.py
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py.bak
├── voltstream_project/
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
└── ...
(всего 67 строк)
```

Рисунок Б.1 – Структура файлов и каталогов проекта *VoltStream*

ПРИЛОЖЕНИЕ Б

(справочное)

Результаты тестирования *REST-API*

В.1 Результаты функционального тестирования

Таблица В.1 – Результаты функционального тестирования *REST-API*

№	Тест-кейс	Запрос	Ожидаемый результат	Фактический результат
1	2	3	4	5
1	Открытие главной страницы редактора	<i>GET /</i>	<i>HTTP 200, HTML</i> страница с заголовком <i>VoltStream</i>	<i>HTTP 200</i> , 45 916 байт, 0,02 с
2	Чтение каталога трансформаторов	<i>GET /get_catalog_nodes/?category=ТП&page=1</i>	<i>JSON</i> со списком трансформаторов	<i>HTTP 200</i> , 78 записей, 0,006 с
3	Чтение каталога ЛЭП	<i>GET /get_catalog_nodes/?category=ЛЭП&page=1</i>	<i>JSON</i> , страница 1 из <i>N</i>	<i>HTTP 200</i> , 50 записей на странице, 0,005 с
4	Чтение реестра папок и схем	<i>GET /api/registry/</i>	<i>JSON</i> со списком папок и схем	<i>HTTP 200</i> , 10 папок и 3 схемы, 0,004 с
5	Список ревизий схем	<i>GET /api/list/</i>	<i>JSON</i> со списком ревизий	<i>HTTP 200</i> , 4 ревизии, 0,004 с

Продолжение таблицы В.1

1	2	3	4	5
6	Загрузка существующей ревизии	<i>GET</i> <i>/api/load/16/</i>	<i>JSON</i> с топологией ревизии	<i>HTTP</i> 200, узлов: 2, линий: 0, 0,004 с
7	Сохранение новой схемы	<i>POST</i> <i>/api/save/</i> (валидный <i>JSON</i>)	<i>HTTP</i> 200, идентификатор ревизии	<i>HTTP</i> 200, <i>id</i> =18, 0,014 с
8	Получение свойств трансформатора	<i>GET</i> <i>/api/get_node_details/?db_id=1</i> & <i>type</i> =ТП	<i>JSON</i> со свойствами объекта	<i>HTTP</i> 200, корректный <i>JSON</i> , 0,004 с
9	Поиск и пагинация каталога	<i>GET</i> <i>/get_catalog_nodes/?category=ТП&page=2&search=ТМ-</i>	<i>JSON</i> , отфильтрованные записи	<i>HTTP</i> 200, 11 записей, 0,005 с
10	Невалидная категория каталога	<i>GET</i> <i>/get_catalog_nodes/?category=ZZZ</i>	<i>HTTP</i> 400, сообщение об ошибке	<i>HTTP</i> 400, {"error": "Invalid category"}
11	Доступ к админ-панели без авторизации	<i>GET</i> <i>/admin-panel/</i>	Редирект на страницу входа	<i>HTTP</i> 302, <i>/admin/login/?next=/admin-panel/</i>
12	Загрузка несуществующей ревизии	<i>GET</i> <i>/api/load/9999/</i>	<i>HTTP</i> 404	<i>HTTP</i> 404, страница «Page not found»

```

PS C:\Users\User\Desktop\Проект> curl -s -i -X POST http://127.0.0.1:8765/api/save/ \
-H "Content-Type: application/json" \
-d '{"name":"Тестовая схема","folder_id":1,"nodes":[...],"lines":[]}'

HTTP/1.1 200 OK
Date: Wed, 29 Apr 2026 14:28:58 GMT
Server: WSGIServer/0.2 CPython/3.12.7
Content-Type: application/json
X-Frame-Options: DENY
Content-Length: 31
Vary: Cookie
X-Content-Type-Options: nosniff
Referrer-Policy: same-origin
Cross-Origin-Opener-Policy: same-origin

{"status": "success", "id": 24}

PS C:\Users\User\Desktop\Проект> _

```

Рисунок В.1 – Скриншот результата теста сохранения схемы (*curl*, *HTTP* 200)

В.2 Результаты модульного (*unit*) тестирования

Таблица В.2 – Состав *unit*-тестов программного продукта

№	Тестовый класс	Метод теста	Что проверяется
1	<i>Save-LoadSchemeTest</i>	<i>test_save_then_load_scheme_creates_revision_and_returns_topology</i>	<i>POST /api/save/</i> создаёт ревизию; <i>GET /api/load/<id>/</i> возвращает ту же топологию
2	<i>CatalogSearchPaginationTest</i>	<i>test_pagination_returns_50_items_on_first_page</i>	Каталог отдаёт 50 записей на странице; корректное число страниц
3	<i>CatalogSearchPaginationTest</i>	<i>test_search_filters_by_mark_substring</i>	Поиск по подстроке возвращает только подходящие записи
4	<i>CatalogSearchPaginationTest</i>	<i>test_invalid_category_returns_400</i>	Невалидная категория приводит к <i>HTTP</i> 400
5	<i>TransformerCrudTest</i>	<i>test_list_returns_existing_transformer</i>	<i>GET /api/admin/transformers/</i> возвращает созданный объект
6	<i>TransformerCrudTest</i>	<i>test_create_persists_new_transformer</i>	<i>POST /api/admin/transformers/create/</i> создаёт новый объект
7	<i>TransformerCrudTest</i>	<i>test_delete_removes_transformer</i>	<i>DELETE</i> удаляет объект из базы


```

PS C:\Users\User\Desktop\Проект> python manage.py test designer -v 2
Creating test database for alias 'default' (...mode=memory&cache=shared)...
Found 7 test(s).
Operations to perform:
  Synchronize unmigrated apps: messages, staticfiles
  Apply all migrations: admin, auth, contenttypes, designer, sessions
Running migrations: ... Applying designer.0009 ... OK

test_invalid_category_returns_400 (designer.tests.CatalogSearchPaginationTest) ... ok
test_pagination_returns_50_items_on_first_page (...CatalogSearchPaginationTest) ... ok
test_search_filters_by_mark_substring (...CatalogSearchPaginationTest) ... ok
test_save_then_load_scheme_creates_revision_and_returns_topology
(designer.tests.SaveLoadSchemeTest) ... ok
test_create_persists_new_transformer (designer.tests.TransformerCrudTest) ... ok
test_delete_removes_transformer (designer.tests.TransformerCrudTest) ... ok
test_list_returns_existing_transformer (designer.tests.TransformerCrudTest) ... ok

-----
Ran 7 tests in 1.486s

OK
Destroying test database for alias 'default'...
System check identified no issues (0 silenced).

PS C:\Users\User\Desktop\Проект> _

```

Рисунок В.2 – Результат запуска модульных тестов (*Ran 7 tests in 1.486s, OK*)

ПРИЛОЖЕНИЕ В

(обязательное)

Листинг программы

A.1 Файл `designer/urls.py`

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.editor_view, name='editor'),
    path('api/save/', views.save_scheme_api, name='save_scheme_api'),
    path('api/list/', views.list_revisions_api, name='list_revisions_api'),
    path('api/load/<int:rev_id>/', views.load_scheme_api, name='load_scheme_api'),
    path('api/load_scheme_by_id/<int:scheme_id>/', views.load_scheme_by_id_api, name='load_scheme_by_id_api'),
    path('get_catalog_nodes/', views.get_catalog_nodes, name='get_catalog_nodes'),
    path('api/get_node_details/', views.get_node_details, name='get_node_details'),
    path('admin-panel/', views.admin_panel_view, name='admin_panel'),
    path('api/admin/create_org/', views.api_create_organization, name='api_create_org'),
    path('api/admin/create_user/', views.api_create_user, name='api_create_user'),
    path('logout/', views.logout_view, name='logout'),

    # =====
    # НОВЫЕ АРІ ДЛЯ УПРАВЛЕНИЯ СПРАВОЧНИКАМИ
    # =====
    path('api/admin/transformers/', views.api_transformers_list, name='api_transformers_list'),
    path('api/admin/transformers/create/', views.api_transformer_create, name='api_transformer_create'),
    path('api/admin/transformers/<int:obj_id>/', views.api_transformer_update, name='api_transformer_update'),
    path('api/admin/transformers/<int:obj_id>/delete/', views.api_transformer_delete, name='api_transformer_delete'),

    path('api/admin/lines/', views.api_lines_list, name='api_lines_list'),
    path('api/admin/lines/create/', views.api_line_create, name='api_line_create'),
    path('api/admin/lines/<int:obj_id>/', views.api_line_update, name='api_line_update'),
    path('api/admin/lines/<int:obj_id>/delete/', views.api_line_delete, name='api_line_delete'),

    path('api/admin/consumers/', views.api_consumers_list, name='api_consumers_list'),
    path('api/admin/consumers/create/', views.api_consumer_create, name='api_consumer_create'),
    path('api/admin/consumers/<int:obj_id>/', views.api_consumer_update, name='api_consumer_update'),
    path('api/admin/consumers/<int:obj_id>/delete/', views.api_consumer_delete, name='api_consumer_delete'),

    path('api/admin/organizations/', views.api_organizations_list, name='api_organizations_list'),
    path('api/admin/organizations/<int:obj_id>/', views.api_organization_update, name='api_organization_update'),
    path('api/admin/organizations/<int:obj_id>/delete/', views.api_organization_delete, name='api_organization_delete'),

    path('api/admin/users/', views.api_users_list, name='api_users_list'),
    path('api/admin/users/<int:obj_id>/', views.api_user_update, name='api_user_update'),
    path('api/admin/users/<int:obj_id>/delete/', views.api_user_delete, name='api_user_delete'),

    # =====
    # РЕЕСТР И СХЕМЫ
    # =====
    path('api/registry/', views.api_get_registry, name='api_get_registry'),
    path('api/admin/folders/create/', views.api_create_folder, name='api_create_folder'),
    path('api/admin/folders/<int:folder_id>/', views.api_edit_folder, name='api_edit_folder'),
```

```

path('api/admin/folders/<int:folder_id>/delete/', views.api_delete_folder, name='api_delete_folder'),

# НОВЫЙ МАРШРУТ ДЛЯ УДАЛЕНИЯ СХЕМ
path('api/admin/schemes/<int:scheme_id>/delete/', views.api_delete_scheme, name='api_delete_scheme'),
]

```

A.2 Файл `designer/models/schemes.py`

```

from django.db import models
from django.contrib.auth.models import User # <-- Импортируем стандартного юзера Django
from .core import Organization

class Folder(models.Model):
    parent = models.ForeignKey('self', on_delete=models.CASCADE, null=True, blank=True, related_name='subfolders',
                               verbose_name="Родительская папка")
    org = models.ForeignKey(Organization, on_delete=models.CASCADE, related_name='folders', verbose_name="Организация")
    name = models.CharField(max_length=255, verbose_name="Имя папки")

    class Meta:
        db_table = 'Folders'
        verbose_name = "Папка"
        verbose_name_plural = "Папки"

class Scheme(models.Model):
    folder = models.ForeignKey(Folder, on_delete=models.CASCADE, related_name='schemes', verbose_name="Папка")
    name = models.CharField(max_length=255, verbose_name="Название схемы")
    description = models.TextField(blank=True, null=True, verbose_name="Описание")
    # Меняем AppUser на User
    created_by = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True, related_name='created_schemes', verbose_name="Создатель схемы")

    class Meta:
        db_table = 'Schemes'
        verbose_name = "Схема"
        verbose_name_plural = "Схемы"

class SchemeRevision(models.Model):
    scheme = models.ForeignKey(Scheme, on_delete=models.CASCADE, related_name='revisions', verbose_name="Схема")
    label = models.CharField(max_length=50, verbose_name="Версия (Метка)")
    topology_data = models.JSONField(verbose_name="Топология (JSON)")
    created_at = models.DateTimeField(auto_now_add=True, verbose_name="Дата создания")
    # Меняем AppUser на User
    created_by = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True, related_name='created_revisions', verbose_name="Создатель версии")

    class Meta:
        db_table = 'Scheme_Revisions'
        verbose_name = "Версия схемы"

```

```
verbose_name_plural = "Версии схем"
```

A.3 Файл *designer/views/base.py*

```
* require_methods – универсальный декоратор ограничения HTTP-методов
* staff_required – декоратор «только админ или суперюзер»
* superuser_required – декоратор «только суперюзер»
* json_error / json_ok – единообразные JSON-ответы
* get_val – безопасное извлечение значения из Django-модели
* CrudView – базовый класс для CRUD-ресурсов справочников (ООП-слой)
"""
```

```
from functools import wraps
import json
```

```
from django.http import JsonResponse
```

```
# ----- #
# Унифицированные JSON-ответы
# ----- #
def json_ok(message: str = "", **extra):
    """Успешный JSON-ответ. Любые keyword-поля попадают в тело."""
    payload = {'status': 'success'}
    if message:
        payload['message'] = message
    payload.update(extra)
    return JsonResponse(payload)

def json_error(message: str, status: int = 400, **extra):
    """Ошибочный JSON-ответ с корректным HTTP-кодом."""
    payload = {'status': 'error', 'message': message}
    payload.update(extra)
    return JsonResponse(payload, status=status)
```

```
# ----- #
# Декораторы проверки прав
# ----- #
def require_methods(*methods):
    """Разрешает доступ только указанным HTTP-методам."""
    methods = tuple(m.upper() for m in methods)

    def decorator(func):
        @wraps(func)
        def wrapper(request, *args, **kwargs):
            if request.method not in methods:
                return json_error('Метод не поддерживается', status=405)
            return func(request, *args, **kwargs)
        return wrapper
    return decorator
```

```
def staff_required(func):
    """Только администраторы или суперюзеры (is_staff / is_superuser)."""
    @wraps(func)
    def wrapper(request, *args, **kwargs):
        if not (request.user.is_staff or request.user.is_superuser):
            return json_error('Доступ запрещен', status=403)
        return func(request, *args, **kwargs)
    return wrapper
```

```
def superuser_required(func):
    """Только суперюзер."""
    @wraps(func)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_superuser:
            return json_error('Только суперюзер имеет доступ', status=403)
        return func(request, *args, **kwargs)
    return wrapper
```

```
# ----- #
```

УТИЛИТЫ

```
# ----- #
```

```
def get_val(obj, field_name):
    """Безопасно возвращает строковое значение поля, либо «-»."""
    val = getattr(obj, field_name, None)
    if val is None or str(val).strip() == "":
        return '-'
    return str(val)
```

```
def parse_json_body(request):
    """Парсит JSON-тело запроса. Возвращает dict или пустой dict."""
    if not request.body:
        return {}
    try:
        return json.loads(request.body)
    except ValueError:
        return {}
```

```
def to_float(value, default=None):
    """Аккуратное приведение к float с учётом пустых строк."""
    if value is None or value == "":
        return default
    try:
        return float(value)
    except (TypeError, ValueError):
        return default
```

```
def to_int(value, default=None):
    """Аккуратное приведение к int с учётом пустых строк."""
    if value is None or value == "":
        return default
    try:
        return int(value)
    except (TypeError, ValueError):
```

```
return default
```

```
# ----- #
# Базовый CRUD-класс
# ----- #
class CrudView:
    """
    Базовый ООР-слой для CRUD справочников (ТП, ЛЭП, Потребители, ...).

    Наследник задаёт:
        model          – Django-модель
        order_by       – поле для сортировки списка
        fields_read     – {json_key: model_attr} для чтения
        fields_write    – {json_key: (model_attr, parser)} для записи

    Методы list/create/update/delete – тонкие обёртки с единой обработкой
    ошибок, правами доступа и JSON-ответами.
    """

    model = None
    order_by = 'id'
    fields_read: dict = {}
    fields_write: dict = {}

    # ----- сериализация ----- #
    @classmethod
    def serialize(cls, obj) -> dict:
        data = {'id': obj.id}
        for key, attr in cls.fields_read.items():
            val = getattr(obj, attr, None)
            data[key] = " if val is None else str(val) if not isinstance(val, str) else val
        return data

    # ----- read ----- #
    @classmethod
    def list(cls, request):
        qs = cls.model.objects.all().order_by(cls.order_by)
        return json_ok(data=[cls.serialize(o) for o in qs])

    # ----- create ----- #
    @classmethod
    def create(cls, request):
        data = parse_json_body(request)
        kwargs = cls._apply_fields({}, data, for_create=True)
        obj = cls.model.objects.create(**kwargs)
        return json_ok(f'{cls.model.__name__} создан', id=obj.id)

    # ----- update ----- #
    @classmethod
    def update(cls, request, obj_id):
        from django.shortcuts import get_object_or_404
        obj = get_object_or_404(cls.model, id=obj_id)
        data = parse_json_body(request)
        for key, (attr, parser) in cls.fields_write.items():
            if key in data:
                setattr(obj, attr, parser(data[key]))
        obj.save()
        return json_ok(f'{cls.model.__name__} обновлён')
```

```

# ----- delete ----- #
@classmethod
def delete(cls, request, obj_id):
    from django.shortcuts import get_object_or_404
    obj = get_object_or_404(cls.model, id=obj_id)
    obj.delete()
    return json_ok(f'{cls.model.__name__} удалён')

# ----- internal ----- #
@classmethod
def _apply_fields(cls, target: dict, data: dict, for_create: bool) -> dict:
    for key, (attr, parser) in cls.fields_write.items():
        if for_create or key in data:
            target[attr] = parser(data.get(key))
    return target

```

A.4 Файл designer/views/schemes.py

```

from django.shortcuts import get_object_or_404
from django.views.decorators.csrf import csrf_exempt
from django.utils import timezone

```

```

from ..models import SchemeRevision, Scheme, Folder, Organization
from .base import (
    json_ok, json_error, require_methods, staff_required, parse_json_body,
)

```

```

# =====
# СОХРАНЕНИЕ / ЗАГРУЗКА ОТДЕЛЬНЫХ СХЕМ
# =====
@csrf_exempt
@require_methods('POST')
def save_scheme_api(request):
    """Сохраняет схему. Привязывает к папке (если передан folder_id)."""
    try:
        data = parse_json_body(request)
        scheme_name = data.get('name', 'Новая схема')
        folder_id = data.get('folder_id')
        topology = {
            'nodes': data.get('nodes', []),
            'lines': data.get('lines', []),
        }

        if folder_id:
            folder = get_object_or_404(Folder, id=folder_id)
        else:
            org, _ = Organization.objects.get_or_create(name='Главная организация')
            folder, _ = Folder.objects.get_or_create(name='Рабочие схемы', org=org)

        scheme, _ = Scheme.objects.get_or_create(name=scheme_name, folder=folder)

        revision = SchemeRevision.objects.create(
            scheme=scheme,

```

```

        label=f'Версия от {timezone.now().strftime("%d.%m.%Y %H:%M')}',
        topology_data=topology,
        created_by=request.user if request.user.is_authenticated else None,
    )
    return json_ok(id=revision.id)
except Exception as e:
    return json_error(str(e))

def list_revisions_api(request):
    """Список всех версий схем (для старой модалки загрузки)."""
    revisions = (
        SchemeRevision.objects.all()
        .order_by('-created_at')
        .values('id', 'label', 'created_at')
    )
    return json_ok(revisions=list(revisions))

def load_scheme_api(request, rev_id):
    """Загрузка конкретной ревизии по её ID."""
    revision = get_object_or_404(SchemeRevision, id=rev_id)
    return json_ok(topology_data=revision.topology_data, label=revision.label)

def load_scheme_by_id_api(request, scheme_id):
    """Загружает последнюю ревизию схемы (клик по схеме в дереве)."""
    scheme = get_object_or_404(Scheme, id=scheme_id)
    last_revision = scheme.revisions.order_by('-created_at').first()

    if not last_revision:
        return json_error('У схемы нет сохранений', status=404)

    return json_ok(
        name=scheme.name,
        topology_data=last_revision.topology_data,
    )

# =====
# РЕЕСТР (ПАПКИ + СХЕМЫ)
# =====
def api_get_registry(request):
    """Возвращает плоский список папок и схем для построения дерева на фронте."""
    folders = list(Folder.objects.values('id', 'name', 'parent_id'))
    schemes = list(Scheme.objects.values('id', 'name', 'folder_id'))
    return json_ok(folders=folders, schemes=schemes)

@csrf_exempt
@require_methods('POST')
@staff_required
def api_create_folder(request):
    """Создаёт новую папку (в том числе дочернюю, если передан parent_id)."""
    data = parse_json_body(request)
    name = data.get('name')
    parent_id = data.get('parent_id')

```



```

if not name:
    return json_error('Имя папки обязательно')

# Привязываем к организации текущего админа (или к первой существующей)
org = None
if hasattr(request.user, 'profile') and request.user.profile.organization:
    org = request.user.profile.organization
else:
    org = Organization.objects.first() or Organization.objects.create(
        name='Главная организация'
    )

parent = Folder.objects.filter(id=parent_id).first() if parent_id else None
Folder.objects.create(name=name, parent=parent, org=org)
return json_ok('Папка успешно создана')

@csrf_exempt
@require_methods('PUT')
@staff_required
def api_edit_folder(request, folder_id):
    """Переименовывает папку и/или меняет её родителя."""
    try:
        folder = get_object_or_404(Folder, id=folder_id)
        data = parse_json_body(request)

        folder.name = data.get('name', folder.name)
        if 'parent_id' in data:
            parent_id = data.get('parent_id')
            folder.parent = (
                Folder.objects.filter(id=parent_id).first() if parent_id else None
            )
        folder.save()
        return json_ok('Папка обновлена')
    except Exception as e:
        return json_error(str(e))

@csrf_exempt
@require_methods('DELETE')
@staff_required
def api_delete_folder(request, folder_id):
    """Удаляет папку и всё её содержимое (каскад настраивается в модели)."""
    try:
        folder = get_object_or_404(Folder, id=folder_id)
        folder.delete()
        return json_ok('Папка удалена')
    except Exception as e:
        return json_error(str(e))

@csrf_exempt
@require_methods('DELETE')
@staff_required
def api_delete_scheme(request, scheme_id):
    """Удаляет схему со всеми её ревизиями."""
    try:
        scheme = get_object_or_404(Scheme, id=scheme_id)

```

```

    scheme.delete()
    return json_ok('Схема удалена')
except Exception as e:
    return json_error(str(e))

```

A.5 Файл *designer/views/admin_lines.py*

```

from django.views.decorators.csrf import csrf_exempt

from ..models import RefLine
from .base import (
    CrudView, json_error, staff_required, superuser_required,
    require_methods, to_float, to_int,
)

class LineCrud(CrudView):
    model = RefLine
    order_by = 'mark'

    fields_read = {
        'mark': 'mark',
        'material': 'material',
        'insulation': 'insulation',
        'cores_count': 'cores_count',
        'cross_section': 'cross_section',
        'r_phase_ohm_km': 'r_phase_ohm_km',
        'r_null_ohm_km': 'r_null_ohm_km',
        'r_add_ohm_km': 'r_add_ohm_km',
    }

    fields_write = {
        'mark': ('mark', lambda v: v or ""),
        'material': ('material', lambda v: v or ""),
        'insulation': ('insulation', lambda v: v or ""),
        'cores_count': ('cores_count', to_int),
        'cross_section': ('cross_section', to_float),
        'r_phase_ohm_km': ('r_phase_ohm_km', to_float),
        'r_null_ohm_km': ('r_null_ohm_km', to_float),
        'r_add_ohm_km': ('r_add_ohm_km', to_float),
    }

    @csrf_exempt
    @staff_required
    def api_lines_list(request):
        return LineCrud.list(request)

    @csrf_exempt
    @require_methods('POST')
    @staff_required
    def api_line_create(request):
        try:

```

```

        return LineCrud.create(request)
    except Exception as e:
        return json_error(str(e))

@csrf_exempt
@require_methods('PUT')
@staff_required
def api_line_update(request, obj_id):
    try:
        return LineCrud.update(request, obj_id)
    except Exception as e:
        return json_error(str(e))

@csrf_exempt
@require_methods('DELETE')
@superuser_required
def api_line_delete(request, obj_id):
    try:
        return LineCrud.delete(request, obj_id)
    except Exception as e:
        return json_error(str(e))

```

A.6 Файл designer/tests.py

```

import json

from django.contrib.auth.models import User
from django.test import TestCase, Client
from django.urls import reverse

from designer.models import (
    Organization, Folder, Scheme, SchemeRevision,
    RefTransformer, RefLine,
)

class SaveLoadSchemeTest(TestCase):
    """ET-1. Проверяет полный цикл сохранение -> загрузка схемы по API."""

    def setUp(self):
        self.client = Client()
        self.org = Organization.objects.create(name="Тестовая организация")
        self.folder = Folder.objects.create(name="Тестовая папка", org=self.org)

    def test_save_then_load_scheme_creates_revision_and_returns_topology(self):
        topology = {
            "name": "Тестовая схема 0.4 кБ",
            "folder_id": self.folder.id,
            "nodes": [
                {"id": 1, "x": 100, "y": 100, "type": "ТП",
                 "name": "ТМ-100", "db_id": 9, "params": {}},
                {"id": 2, "x": 300, "y": 100, "type": "Абонент",
                 "name": "Дом 1", "db_id": 1, "params": {}},
            ]
        }

```

```

    ],
    "lines": [
        {"id": 3, "start_node_id": 1, "end_node_id": 2,
         "db_id": 1, "params": {}},
    ],
}

save_resp = self.client.post(
    "/api/save/",
    data=json.dumps(topology),
    content_type="application/json",
)

self.assertEqual(save_resp.status_code, 200)
save_body = save_resp.json()
self.assertEqual(save_body["status"], "success")
rev_id = save_body["id"]

# Ревизия и схема действительно появились в БД.
self.assertTrue(SchemeRevision.objects.filter(id=rev_id).exists())
self.assertTrue(
    Scheme.objects.filter(name="Тестовая схема 0.4 кВ").exists()
)

# Топология на загрузке совпадает с сохранённой.
load_resp = self.client.get(f"/api/load/{rev_id}/")
self.assertEqual(load_resp.status_code, 200)
load_body = load_resp.json()
self.assertEqual(load_body["status"], "success")
self.assertEqual(len(load_body["topology_data"]["nodes"]), 2)
self.assertEqual(len(load_body["topology_data"]["lines"]), 1)
self.assertEqual(
    load_body["topology_data"]["nodes"][0]["name"], "ТМ-100"
)

class CatalogSearchPaginationTest(TestCase):
    """ЕТ-2. Проверяет пагинацию и поиск в каталоге ЛЭП."""

    def setUp(self):
        self.client = Client()
        for i in range(60):
            RefLine.objects.create(
                mark=f"САПГ {i}x25",
                material="Алюминий",
                cores_count=4,
                cross_section=25.0,
                r_phase_ohm_km=1.20,
            )
            RefLine.objects.create(
                mark="АС-50", material="Алюмосталь",
                cores_count=1, cross_section=50.0, r_phase_ohm_km=0.6,
            )

    def test_pagination_returns_50_items_on_first_page(self):
        resp = self.client.get("/get_catalog_nodes/?category=ЛЭП&page=1")
        self.assertEqual(resp.status_code, 200)
        body = resp.json()
        # Пагинация выдаёт 50 элементов на страницу.
        self.assertEqual(len(body["items"]), 50)

```

```

self.assertEqual(body["current_page"], 1)
self.assertEqual(body["total_pages"], 2)

def test_search_filters_by_mark_substring(self):
    resp = self.client.get(
        "/get_catalog_nodes/?category=ЛЭП&page=1&search=AC"
    )
    self.assertEqual(resp.status_code, 200)
    body = resp.json()
    self.assertEqual(len(body["items"]), 1)
    self.assertEqual(body["items"][0]["name"], "AC-50")

def test_invalid_category_returns_400(self):
    resp = self.client.get("/get_catalog_nodes/?category=ZZZ&page=1")
    self.assertEqual(resp.status_code, 400)
    self.assertIn("Invalid category", resp.json()["error"])

class TransformerCrudTest(TestCase):
    """ET-3. Проверяет CRUD-эндпоинты справочника трансформаторов."""

    def setUp(self):
        self.client = Client()
        self.admin = User.objects.create_superuser(
            "test_admin", "admin@example.com", "passw0rd"
        )
        self.client.force_login(self.admin)
        self.transformer = RefTransformer.objects.create(
            type_name="TM-160",
            nominal_power=160.0,
            voltage_hv=10.0,
            voltage_lv=0.4,
        )

    def test_list_returns_existing_transformer(self):
        resp = self.client.get("/api/admin/transformers/")
        self.assertEqual(resp.status_code, 200)
        body = resp.json()
        self.assertEqual(body["status"], "success")
        names = [item["mark"] if "mark" in item else item.get("type_name")
                  for item in body["data"]]
        # Поле type_name присутствует и совпадает с созданным объектом.
        self.assertIn("TM-160", names)

    def test_create_persists_new_transformer(self):
        payload = {
            "type_name": "ТМГ-630",
            "nominal_power": 630.0,
            "losses_no_load": 1.05,
            "losses_short_circuit": 7.6,
            "voltage_hv": 10.0,
            "voltage_lv": 0.4,
            "voltage_short_circuit_pct": 5.5,
            "pbv_stages_count": 5,
            "pbv_step_pct": 2.5,
        }
        resp = self.client.post(
            "/api/admin/transformers/create/",
            data=json.dumps(payload),
            content_type="application/json",

```

```

    )
    self.assertEqual(resp.status_code, 200)
    self.assertEqual(resp.json()["status"], "success")
    self.assertTrue(
        RefTransformer.objects.filter(type_name="TMΓ-630").exists()
    )

def test_delete_removes_transformer(self):
    url = f"/api/admin/transformers/{self.transformer.id}/delete/"
    resp = self.client.delete(url)
    self.assertEqual(resp.status_code, 200)
    self.assertEqual(resp.json()["status"], "success")
    self.assertFalse(
        RefTransformer.objects.filter(id=self.transformer.id).exists()
    )

```